National Tsing Hua University
Department of Electrical Engineering
EE429200 IC Design Laboratory, Fall 2025

Homework Assignment #2 **(10%, total 100 points)**
Enigma
Assigned on Oct. 2, 2025
Due by **Oct. 16, 2025**

Assignment Description

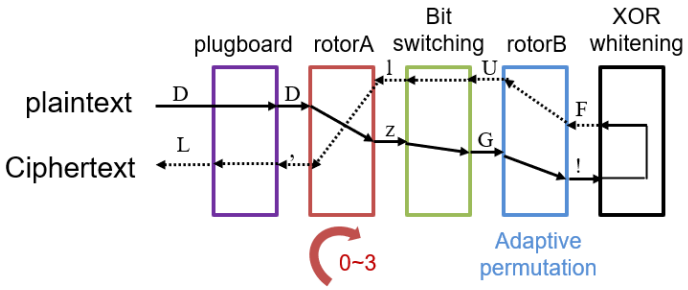


Fig. 1. An Enigma with two rotors and a plugboard. '!' is the input plaintext and 'L' is the output ciphertext.

Enigma is the electro-mechanical rotor cipher machine used by the German military in World War II.

As shown in Fig. 1, Enigma generates one 6-bit ciphertext from one 6-bit input plaintext. A plaintext goes through the plugboard, the rotors, the bit-switching box, and the XOR whitening. Then, it reverses the path, passing through the same components to become a ciphertext. In this assignment, you need to implement a version of Enigma using Verilog RTL.

A rotor, or a plugboard is essentially a mapping mechanism that maps a 6-bit input to a 6-bit output. As shown in Fig. 1, rotorA maps 'E' into 'Z' in the **forward** pass (solid line), and maps 'z' into 'D' in the **inverse** pass (dashed line). An example of rotorA is shown in Fig. 2. In the forward pass, the input is an entry of rotorA (E: '0x24'), and the output is the symbol of that entry (Z: '0x39'). In the inverse pass, the input is a symbol (z: '0x19'), and the output is the address of the entry that contains the symbol (D: '0x23'). Fig. 3 shows the full encryption process that encrypts 'z' into '&'.

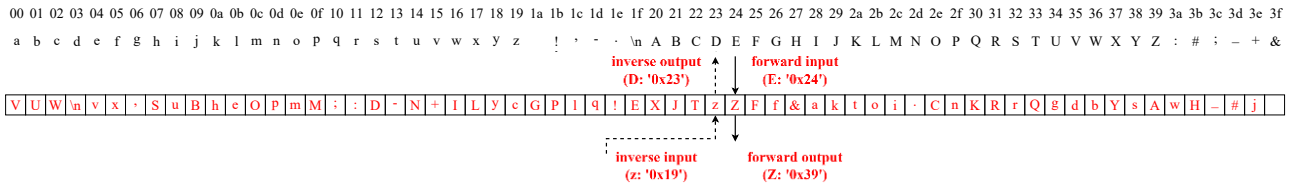


Fig. 2. Example of a rotor.

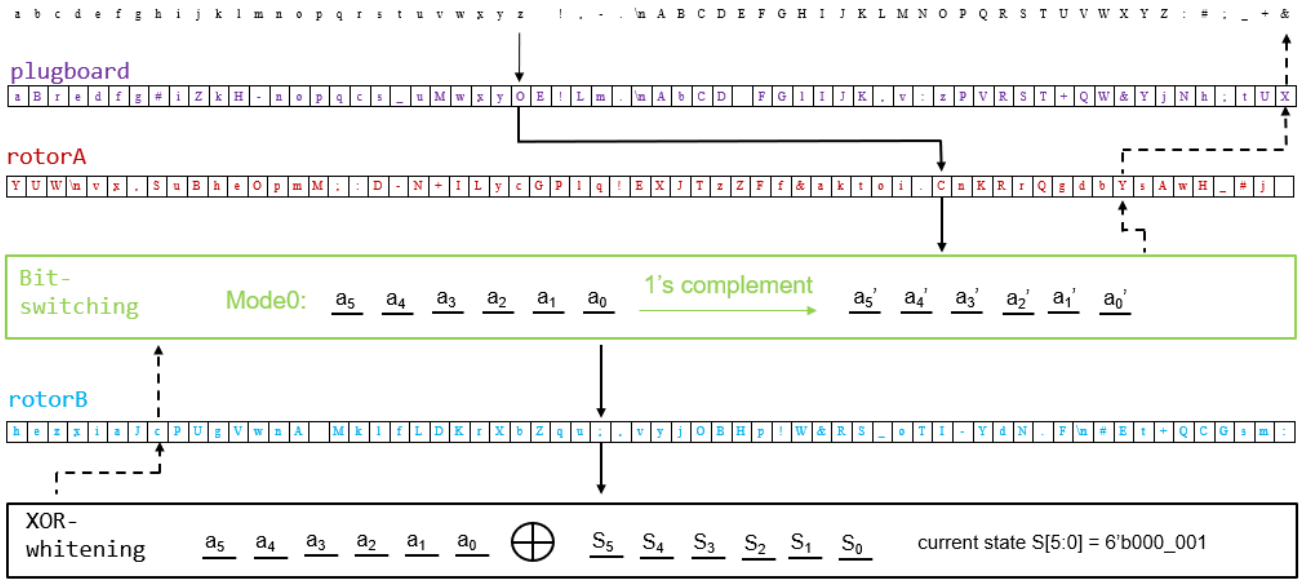


Fig. 3. Example of encrypting 'z' to '&'.

Ciphertext would be easy to crack if the rotor never moves and provides a fixed mapping. To improve the security of Enigma, bit-switching box changes modes and a rotor rotates or permutes for each text it encrypts or decrypts.

1. Plugboard

The plugboard provides additional security to Enigma. A connection on the plugboard swaps two symbols. As shown in Fig. 4, 'E' and 'O' are connected, so the plugboard outputs 'O' if the input is 'E', and vice versa. If a symbol is not plugged, the plugboard outputs the same symbol. Note that the configuration of the plugboard changes daily to make the ciphertext harder to crack in the real world.

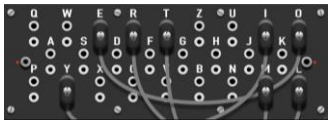


Fig. 4. Illustration of a plugboard. Ref: http://www.terrylong.org/images/screenshots/2.png

In this assignment, there are 16 connections in the plugboard, so 32 symbols are plugged and 32 symbols are not plugged. One example of the plugboard is shown in Fig. 5. The configuration of the plugboard is loaded before encryption or decryption.

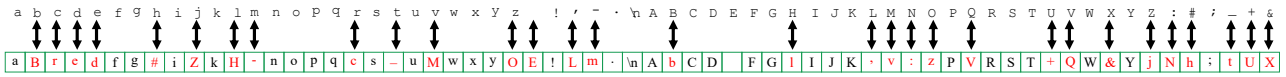


Fig. 5. Example of a plugboard. The 16 arrows indicate the 16 connections between symbols.

2. Bit-switching box

The bit-switching box uses four modes for encryption/decryption. When the first text enters the Enigma, it defaults to Mode 0. The mode for the next text is determined based on either **output of the forward process during encryption** or **input of the inverse process during decryption**. For bit switching operations in different modes, please refer to Fig. 6.

Assuming a 6-bit output {a5, a4, a3, a2, a1, a0} is obtained during encryption, the mode for the next bit-switching box operation is determined by the last two bits {a1, a0}. For example, if 'Z' = 0x39, which is represented as 6'b11_10_01 in binary, the **least significant** 2-bits are 2'b01. This indicates that the bit-switching box should be switched to Mode 1 before the next encryption.

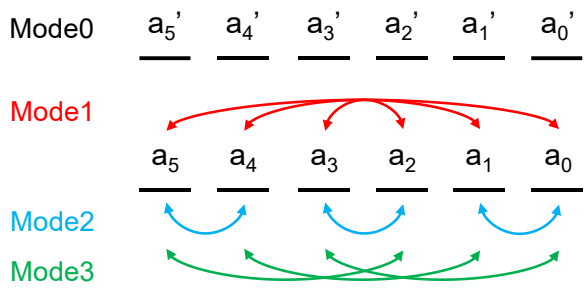
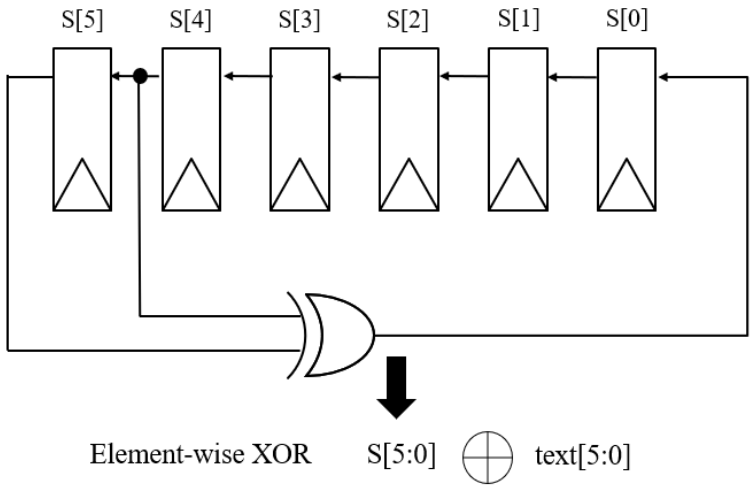


Fig. 6. Bit-switching mode explanation.

$\{a_1, a_0\}$	**Next encryption / decryption**
2'b00	1's complement (Default in the 1st round)
2'b01	Bit-switching by mode1
2'b10	Bit-switching by mode2
2'b11	Bit-switching by mode3

3. XOR-whitening

XOR-whitening perform bit-wise XOR on input text with current state S. We use a LFSR (Linear Feedback Shift Register) to decide what current state is, and update state after XORed with the input text. In this homework, LFSR is expressed as $x^6 + x^5 + 1$ and is shown in Fig. 7 with its first 10 states.



cycle	S[5:0]
1	000001
2	000010
3	000100
4	001000
5	010000
6	100001
7	000011
8	000110
9	001100
10	011000

Fig. 7 XOR-whitening, consisting a 6-bit LFSR with its first ten states.

A 6-bit LFSR consists of 6 Flip-Flops that hold the current state S[5:0]. The feedback is generated by XORing the outputs of S[5] and S[4], and this feedback bit is connected back into S[0] (least significant bit).

Initially, the state registers are reset as **6'b000_001** when srst_n is low. During encryption or decryption, input text is XORed element-wise with current state S[5:0]. After this XOR operation, state registers shift left: S[1]<-S[0], …, S[5]<-S[4] and S[0] is updated with feedback bit. Note that the registers shift left **only when** input text is XORed with current state.

4. RotorA

Fig. 8 shows how the rotorA rotates after Enigma encrypts or decrypts one text. RotorA shifts 0-3 symbols after encrypting one symbol (one forward pass and one inverse pass). In the encryption mode, the amount of rotation is the **least significant** 2-bit of the output of the **forward** pass. In the decryption mode, the amount of rotation is the **least significant** 2-bit of the input of the **inverse** pass.

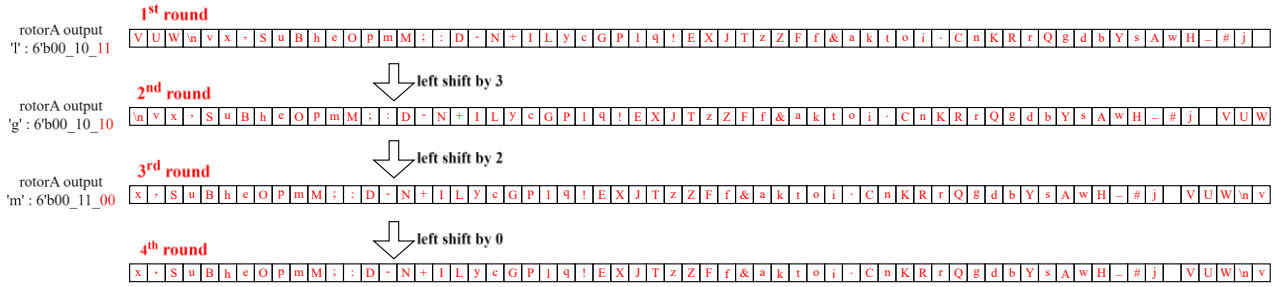


Fig. 8. Rotation of rotorA in the **encryption** mode. The amount of rotation is the least significant 2-bit of the output.

5. RotorB

Fig. 9 shows how the rotorB permutes after Enigma encrypts or decrypts one text. RotorB first goes through the **S-box4** that permutes every four contiguous symbols. Then it goes through a fixed permutation.

There are four permutation modes for the **S-box4** as shown in Fig. 10. In the **encryption** mode, the permutation mode is the **least significant** 2-bit of the output of the **forward** pass. In the **decryption** mode, the permutation mode is the **least significant** 2-bit of the input of the **inverse** pass. After the S-box4, rotorB permutes according to a fixed connection. The detailed connection is shown in Fig. 11.

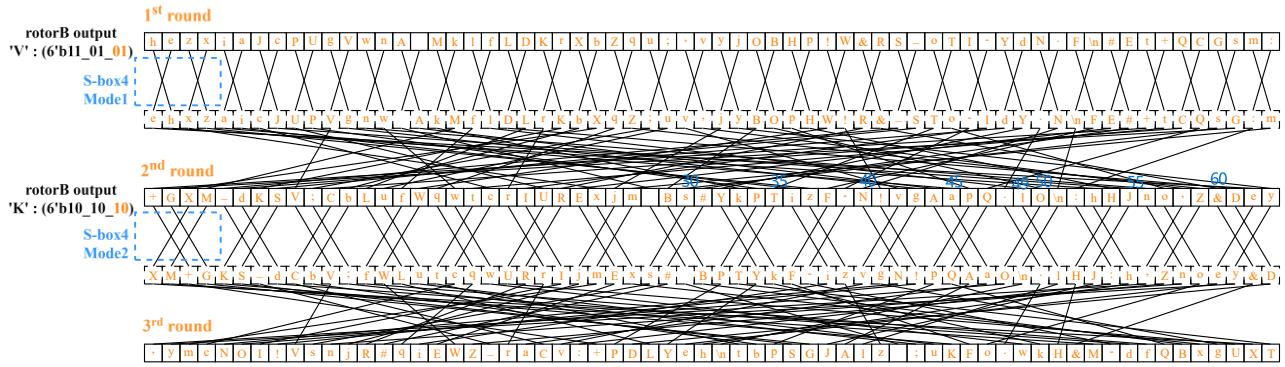


Fig. 9. Permutation of rotorB.

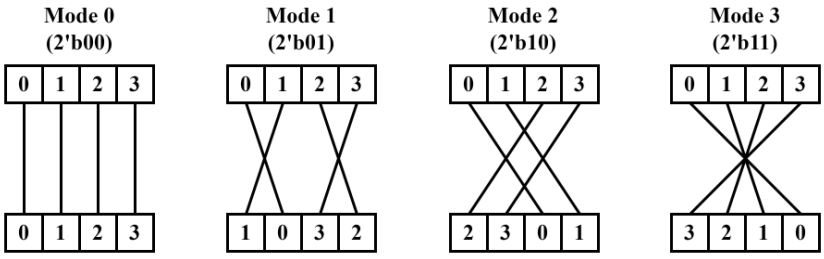


Fig. 10. Pattern of permutations of S-box4.

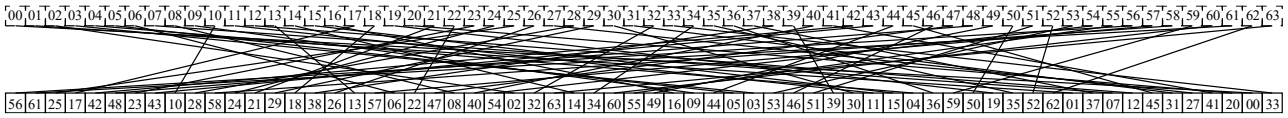


Fig. 11. Pattern of the fixed permutation.

Steps in this homework

There are four parts in this homework. You need to implement rotorA + XOR-whitening in part 1, rotorA + bit-switching + XOR-whitening in part 2, plugboard + rotorA + bit-switching + XOR-whitening in part 3, and plugboard + rotorA + bit-switching + rotorB+ XOR-whitening in part4 as shown in Fig. 12. Besides synthesizable RTLs, you also need to write your own testbench in every part. The system diagram is shown in Fig. 13, and the description of each signal is summarized in Table 1.

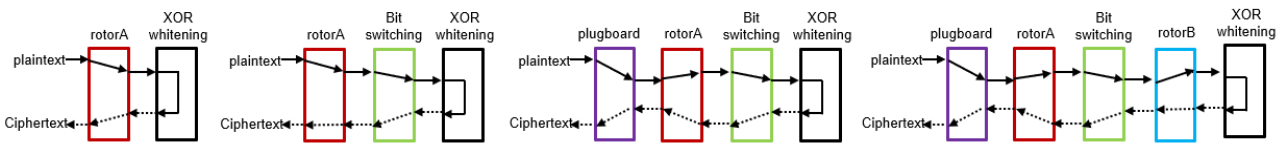


Fig. 12. Enigma part1-4 (left to right).

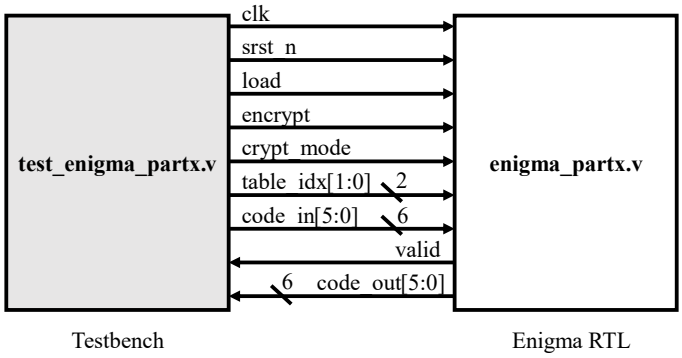


Fig. 13. System diagram

Signal	Description
clk	Clock signal
srst_n	Synchronous reset, active low
load	If load is 1'b1, load **code_in** to rotor or plugboard
encrypt	If encrypt is 1'b1, Enigma encrypt **code_in** to **code_out**
crypt_mode	If crypt_mode is 1'b0, **code_in** is plaintext and **code_out** is ciphertext If crypt_mode is 1'b1, **code_in** is ciphertext and **code_out** is plaintext
table_idx[1:0]	If **table_idx** is 2'b00, load **code_in** to rotorA If **table_idx** is 2'b01, load **code_in** to plugboard If **table_idx** is 2'b10, load **code_in** to rotorB
code_in[5:0]	Input code Plaintext if **crypt_mode** is 1'b0 Ciphertext if **crypt_mode** is 1'b1
valid	**valid** is 1'b1 if **code_out** is valid Otherwise, valid is 1'b0
code_out[5:0]	Output code Ciphertext if **crypt_mode** is 1'b0 Plaintext if **crypt_mode** is 1'b1

Table 1. Description of signals

Part 1

For part 1, you need to complete the RTL code **part1/hdl/enigma_part1.v**, and the testbench **part1/sim/test_enigma_part1.v**. A behavior model (**part1/hd1/behavior_model.v**) of Enigma is provided for your reference. Files for part 1 are summarized in Table 2.

Directory	File	Description
part1/sim	test_enigma_part1.v	Testbench

	rtl_sim.f	File list that includes required Verilog files
	run_rtl_sim.sh	Commands for running VCS simulation
part1/sim/rotor	rotorA.dat	Configuration of rotorA
part1/sim/pat	plaintexts1_part1.dat	Pattern 1
	ciphertexts1_part1.dat	
	plaintexts2_part1.dat	Pattern 2
	ciphertexts2_part1.dat	
part1/hdl	enigma_part1.v	RTL of Enigma for this part
	behavior_model.v	Behavior model of Enigma for your reference

Table 2. Description of files in part 1.

You must pass all four simulations in **run_rtl_sim.sh** to get the full score. Usage of the macros is summarized in Table 3. Also, you must run Spyglass to verify the RTL is synthesizable.

```
vcs -R +v2k -full64 -f sim.f +define+ENCRYPT +define+PAT1 +define+PAT_LEN=29 -debug_access+all -l vcs_encrypt_pat1.log
vcs -R +v2k -full64 -f sim.f +define+DECRYPT +define+PAT1 +define+PAT_LEN=29 -debug_access+all -l vcs_decrypt_pat1.log
vcs -R +v2k -full64 -f sim.f +define+ENCRYPT +define+PAT2 +define+PAT_LEN=111 -debug_access+all -l vcs_encrypt_pat2.log
vcs -R +v2k -full64 -f sim.f +define+DECRYPT +define+PAT2 +define+PAT_LEN=111 -debug_access+all -l vcs_decrypt_pat2.log
```

Fig. 14 run_rtl_sim.sh

Macro	Description
ENCRYPT	crypt_mode = 1'b0 plaintexts.dat as input ciphertexts.dat as golden
DECRYPT	crypt_mode = 1'b1 ciphertexts.dat as input plaintexts.dat as golden
PAT1	Use plaintexts1.dat/ciphertext1.dat
PAT2	Use plaintexts2.dat/ciphertext2.dat
PAT3	Use plaintexts3.dat/ciphertext3.dat
ASCII	Used in **part 3**. If *`ASCII* is defined, the testbench saves the decrypted result to a text file in ASCII format

Table 3. Description of macros used in the simulation.

The reference waveform of part 1 is shown in Fig. 15 and Fig. 16.

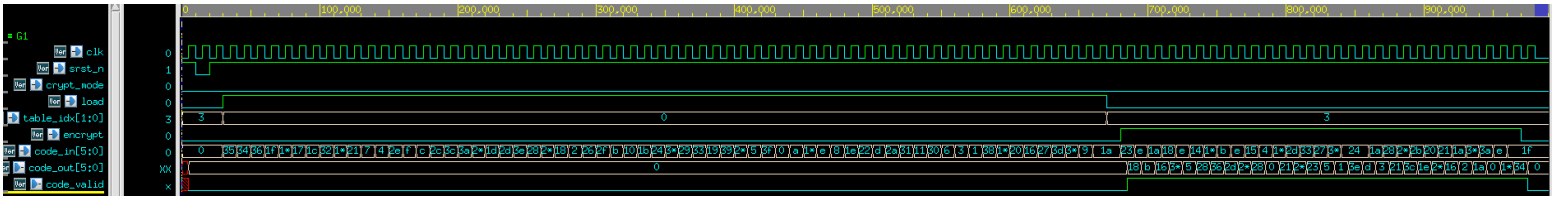


Fig. 15. Reference waveform of encrypting pattern 1. The crypt_mode stays at 1'b0. It takes 64 cycles to load rotorA before encrypting.

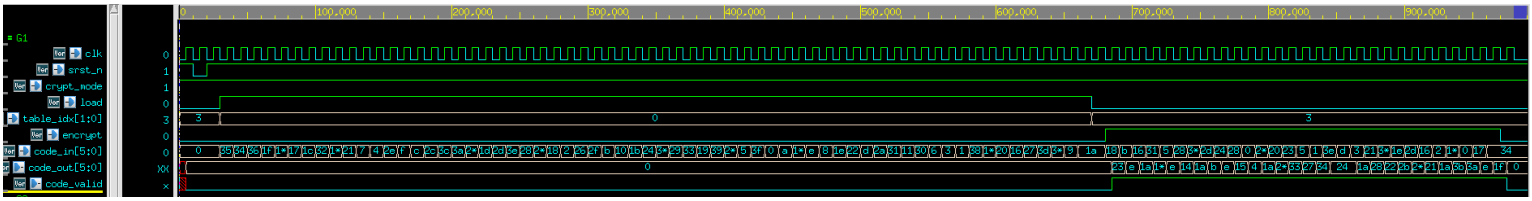


Fig.16. Reference waveform of decrypting pattern 1. The crypt_mode stays at 1'b1.

Part 2

For part 2, you need to complete the RTL code **part2/hdl/enigma_part2.v**, and the testbench **part2/sim/test_enigma_part2.v**. Files for part 2 are summarized in Table 4. You also have to pass all simulations in **run_rtl_sim.sh**, and use Spyglass to verify the RTL is synthesizable.

Directory	File	Description
part2/sim	test_enigma_part2.v	Testbench
	rtl_sim.f	File list that includes required Verilog files
	run_rtl_sim.sh	Commands for running VCS simulation
part2/sim/rotor	rotorA.dat	Configuration of rotorA
part2/sim/pat	plaintexts1_part2.dat	Pattern 1
	ciphertexts1_part2.dat	
	plaintexts2_part2.dat	Pattern 2
	ciphertexts2_part2.dat	
part2/hdl	enigma_part2.v	RTL of Enigma for this part

Table 4. Description of files in part 2.

The reference waveform of part 2 is shown in Fig. 17 and Fig. 18.

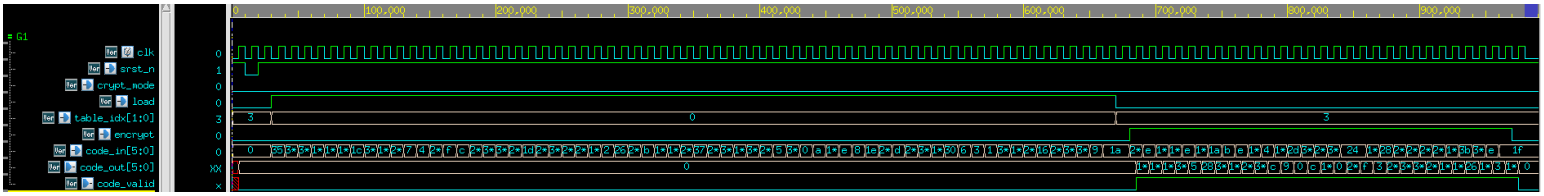


Fig.17. Reference waveform of encrypting pattern 1. The crypt_mode stays at 1'b0. It takes 64 cycles to load rotorA before encrypting.

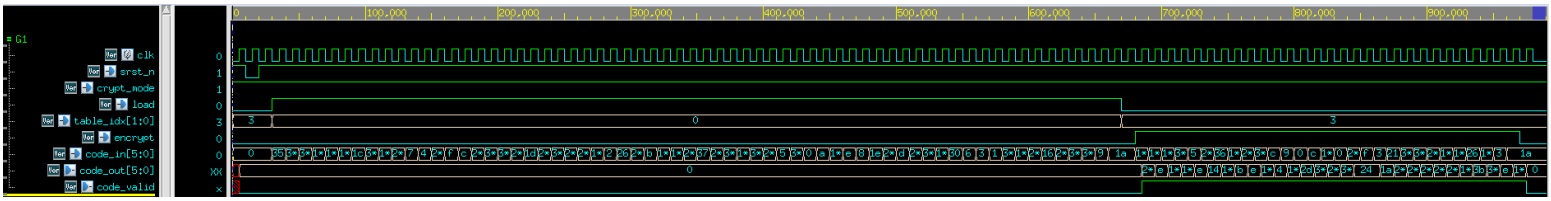


Fig.18. Reference waveform of decrypting pattern 1. The crypt_mode stays at 1'b1.

Part 3

For part 3, you need to complete the RTL code **part3/hdl/enigma_part3.v**, and the testbench **part3/sim/test_enigma_part3.v**. Files for part 3 are summarized in Table 5. You also have to pass all simulations in **run_rtl_sim.sh**, and use Spyglass to verify the RTL is synthesizable.

Directory	File	Description
part3/sim	test_enigma_part3.v	Testbench

	rtl_sim.f	File list that includes required Verilog files
	run_rtl_sim.sh	Commands for running VCS simulation
part3/sim/rotor	plugboard.dat	Configuration of plugboard
	rotorA.dat	Configuration of rotorA
part3/sim/pat	plaintexts1_part3.dat	Pattern 1
	ciphertexts1_part3.dat	
	plaintexts2_part3.dat	Pattern 2
	ciphertexts2_part3.dat	
part3/hdl	enigma_part3.v	RTL of Enigma for this part

Table 5. Description of files in part 3.

The reference waveform of part 3 is shown in Fig. 19 and Fig. 20.

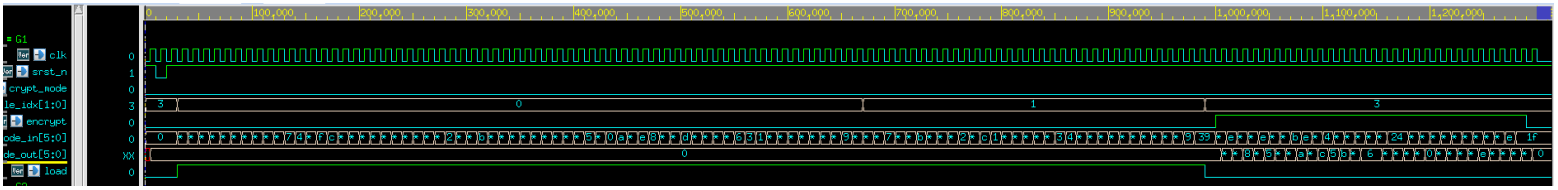


Fig.19. Reference waveform of encrypting pattern 1. The crypt_mode stays at 1'b0. It takes (64+32) cycles to load plugboard and rotorA before encrypting.

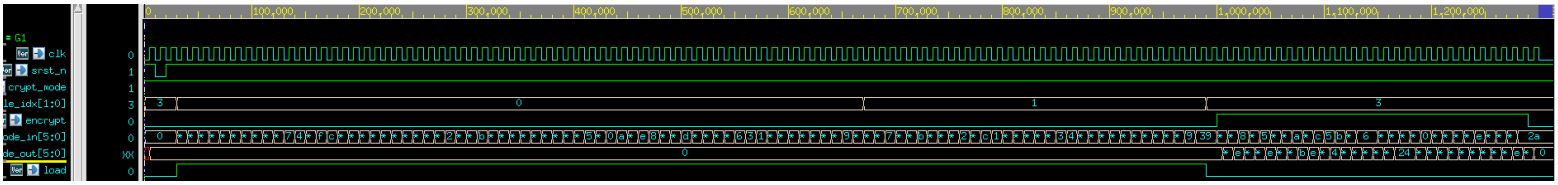


Fig.20. Reference waveform of decrypting pattern 1. The crypt_mode stays at 1'b1.

Part 4

For part 4, you need to complete the RTL code **part4/hdl/enigma_part4.v**, and the testbench **part4/sim/test_enigma_part4.v**. Files for part 4 are summarized in Table 6. You also have to pass all simulations in **run_rtl_sim.sh**, and use Spyglass to verify the RTL is synthesizable.

Directory	File	Description

part4/sim	test_enigma_part4.v	Testbench
	rtl_sim.f	File list that includes required Verilog files
	run_rtl_sim.sh	Commands for running VCS simulation
part4/sim/rotor	rotorA.dat	Configuration of rotorA
	rotorB.dat	Configuration of rotorB
	plugboard.dat	Configuration of plugboard
part4/sim/pat	plaintexts1_part4.dat	Pattern 1
	ciphertexts1_part4.dat	
	plaintexts2_part4.dat	Pattern 2
	ciphertexts2_part4.dat	
	plaintexts3_part4.dat	Pattern 3
	ciphertexts3_part4.dat	
part4/sim/result	plaintexts1_ascii.dat	Decrypted result of ciphertext1 in ASCII format
	plaintexts2_ascii.dat	Decrypted result of ciphertext2 in ASCII format
	plaintexts3_ascii.dat	Decrypted result of ciphertext3 in ASCII format
part4/hdl	enigma_part4.v	RTL of Enigma for this part

Table 6. Description of files in part 4.

An additional pattern (ciphertexts3.dat) is included. For this pattern, the macro *+define+ASCII* is used in the simulation. Your testbench should write the result to part4/sim/result in ASCII format. The result is ASCII art. You can use a text editor on a black background to view it. Check https://en.wikipedia.org/wiki/ASCII_art for more information about ASCII art.

The reference waveform of part 4 is shown in Fig. 21 and Fig 22.

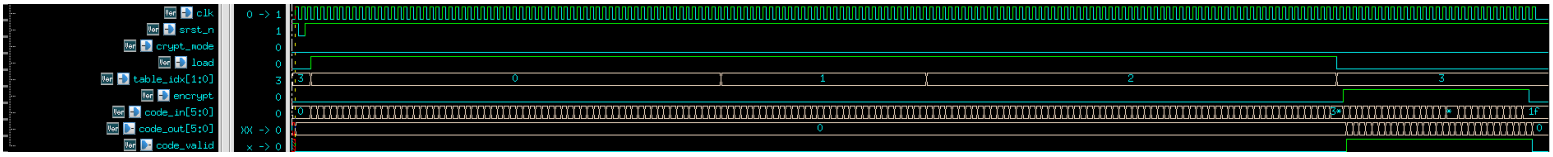


Fig.21. Reference waveform of encrypting pattern 1. The crypt_mode stays at 1'b0. It takes (64+64+32) cycles to load plugboard, rotorA, and rotorB and connections before encrypting.

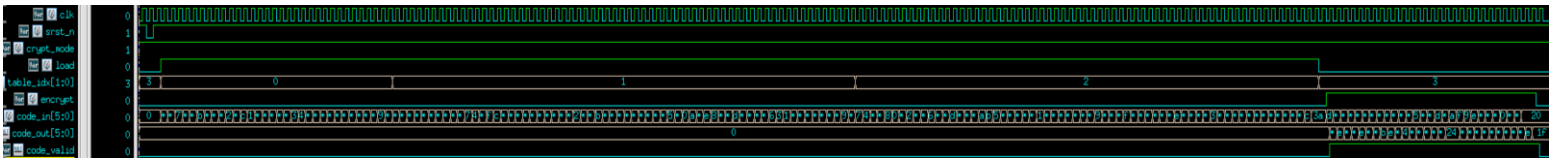


Fig.22. Reference waveform of decrypting pattern 1. The crypt_mode stays at 1'b1.

Grading policy

For all four parts, the enigma should encrypt or decrypt one symbol per cycle after valid pulls up to high. Otherwise, you cannot get the full score. Also, you should not implement your enigma based on the given configuration of rotorA, rotorB and plugboard.dat. TA will use hidden configurations to examine your design, including changing the order of table_idx in part4.

1. (10 points) The functional module enigma_part1.v and the Spyglass report. enigma_part1.v should pass all simulations and the report must show the RTL is synthesizable. Otherwise, you cannot get the score.
2. (10 points) The testbench test_enigma_part1.v. The testbench can test pattern 1 and pattern 2 in both crypt_mode. Note that you should load the rotorA from the first line to the last line of rotorA.dat. TA's testbench will use this order to examine your design.
3. (10 points) The functional module enigma_part2.v and the Spyglass report. enigma_part2.v should pass all simulations and the report must show the RTL is synthesizable. Otherwise, you cannot get the score.
4. (10 points) The testbench test_enigma_part2.v. The testbench can test pattern 1 and pattern 2 in both crypt_mode. Note that you should load the rotorA from the first line to the last line of rotorA.dat. TA's testbench will use this order to examine your design.
5. (10 points) The functional module enigma_part3.v and the Spyglass report. enigma_part3.v should pass all simulations and the report must show the RTL is synthesizable. Otherwise, you cannot get the score.
6. (10 points) The testbench test_enigma_part3.v. The testbench can test pattern 1 and pattern 2 in both crypt_mode. Note that you should load the plugboard, rotorA from the first line to the last line of plugboard.dat and rotorA.dat. TA's testbench will use this order to examine your design.
7. (20 points) The functional module enigma_part4.v and the Spyglass report. enigma_part4.v should pass all simulations and the report must show the RTL is synthesizable. Otherwise, you cannot get the score. Note that you should load the plugboard, rotorA, and rotorB from the first line to the last line of plugboard.dat, rotorA.dat, and rotorB.dat. TA's testbench will use this order to examine your design.
8. (20 points) The testbench test_enigma_part4.v and plaintexts{1,2,3}_ascii.dat. The testbench can test pattern 1 and pattern 2 in both crypt_mode, and pattern 3 in decrypt mode. The testbench should write the result to plaintexts{1,2,3}_ascii.dat in ASCII if the macro *ASCII* is defined.

TA will execute the shell script (.sh) in sim/ to evaluate your homework in each part.

Deliverable

File organization of deliverable is shown in Fig. 23.

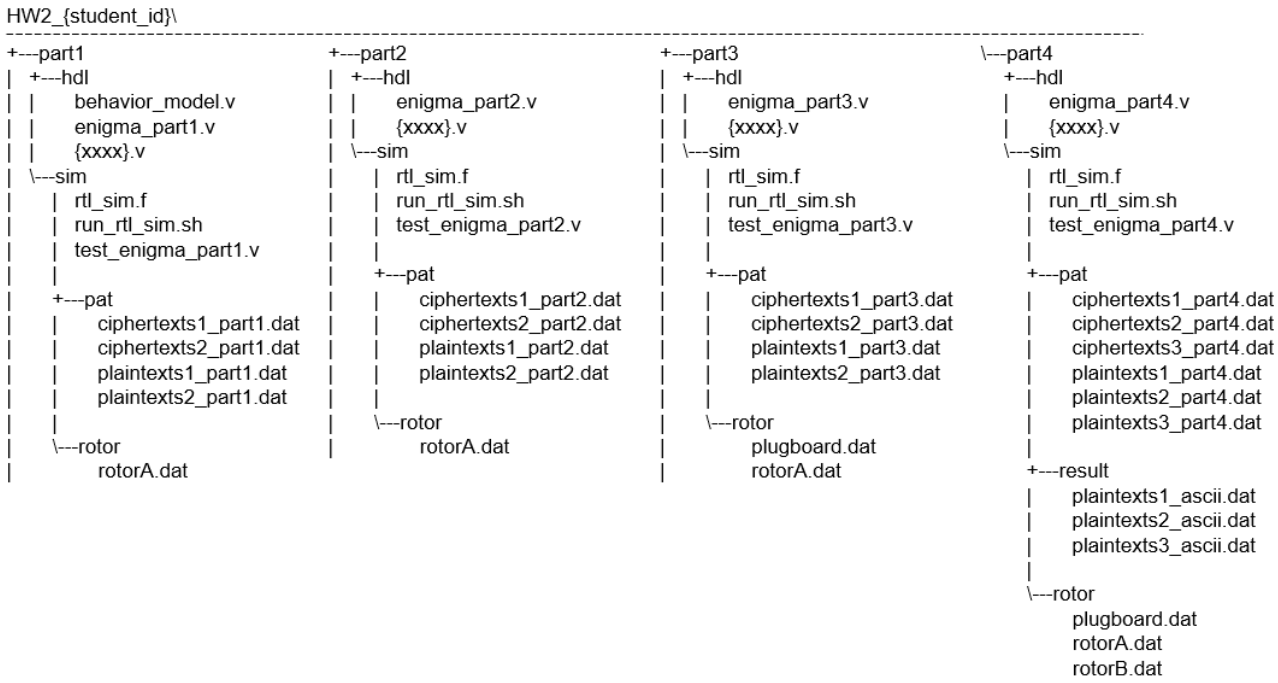


Fig.23. File organization of deliverable.

Note

If your student ID is 123456789, HW2_{student_id} denotes HW2_123456789.

Compress your whole folder HW2_{student_id} into HW2_{student_id}.zip and submit to **eeclass**.

10 points deducted from this homework for wrong file delivery, wrong file organization, or wrong file naming.